Mechanisation Guides Design: Higher-Level Supercompilation via Sequent Calculus

Anonymous

Abstract. A mechanised correspondence between supercompilation and cyclic sequent proof search (driving = cut elimination, generalisation = cut introduction) leads to two productive extensions.

First, identifying generalisation as cut introduction suggests an untrusted oracle architecture: an LLM proposes cut formulas, and the kernel's existing trace-condition check validates them. Soundness is unconditional — the oracle can be replaced by any heuristic without affecting the CIU theorem. A controlled scrambling experiment shows the LLM succeeds on 6 programs even with all function names randomised, providing evidence of compositional reasoning from definitions rather than name recognition.

Second, strengthened induction (the ω-rule) is implemented as a lemma environment: auxiliary lemmas are proved by a sub-supercompiler and used as rewrite rules in the main proof. This enables compiler soundness and reverse-append distribution — properties standard SC cannot prove automatically.

Third, type-index comparison prunes impossible branches from dependent-type case-splits, making programs with different dead branches produce identical residuals.

The system proves 63 primary theorems (81 including infrastructure) with 0 Admitted via `vm_compute`. We report honestly: the LLM lemma proposal loop works in Python but is not yet wired through OCaml extraction; the 6-program scrambling experiment is evidence, not a statistical evaluation.

1 Introduction

Supercompilation [8] fuses composed functions, eliminates intermediate data structures, and discovers recursive structure automatically. Prior work established that standard supercompilation corresponds to focused cyclic sequent proof search: driving is cut elimination, generalisation is cut introduction, and the trace condition is the global progress check of cyclic proof theory.[1] This paper shows that the correspondence is more than a correctness proof — it is a *design instrument* that identified two productive extensions to supercompilation.

[1] A companion mechanisation (under review) formalises this correspondence in Rocq with 0 axioms; the present paper uses the resulting infrastructure as its starting point.

Extension 1: LLM oracle for cut introduction. Syntactic anti-unification is a heuristic for cut introduction, but it is purely structural and cannot discover semantic identities (map fusion, accumulator introduction). The sequent formulation suggests an *oracle architecture*: any function that proposes cut formulas is valid if the kernel accepts the resulting proof graph. We instantiate this with an LLM (DeepSeek V4 Pro); the oracle is untrusted for soundness — the CIU theorem requires only the trace condition check.

Extension 2: LLM lemma proposal for the ω-rule. Properties like $\mathsf{sorted}\ (\mathsf{sort}\ l) = \mathsf{true}$ require auxiliary lemmas. The sequent formulation identifies this as the ω-rule: prove a lemma first, then use it as a rewrite rule. We add a lemma environment to the SC: the LLM proposes auxiliary lemmas, a sub-SC run proves them, and the main SC uses them as rewrite rules. This enables compiler soundness and reverse-append distribution — properties unreachable by standard SC.

Extension 3: Dependent-type branch elimination. The SC's type annotations carry structural information. We prune impossible constructors from case-splits using type-index comparison (e.g., `vnil` eliminated from $\mathsf{Vec}\ (\mathsf{succ}\ n)$). This makes two programs with different dead branches produce identical residuals, enabling more folding opportunities.

What is and is not claimed. The end-to-end pipeline is demonstrated: the LLM proposes a lemma, the sub-SC validates it, and the SC uses it as a rewrite rule. The scrambling experiment (Section 8) uses 6 programs and 24 tests — enough to refute the hypothesis that the LLM relies solely on name recognition, but not a statistical evaluation. The type-index trace condition extension (§3) is specified and implemented but does not yet enable any theorem that was previously out of reach.

Contributions.

1. **LLM oracle architecture** (Section 4). A 2-layer generalisation strategy: anti-unification → LLM oracle. All LLM proposals are validated by the kernel. The scrambling experiment shows the LLM succeeds on 6 programs even with all names scrambled, and with definitions provided, all 6 pass with high confidence.
2. **Mechanised ω-rule** (Section 7). A lemma environment: lemmas are proved by sub-SC and used as rewrite rules in the main SC. This enables compiler correctness and reverse-append distribution. The LLM proposal step succeeds on all tested examples (Section 8).
3. **Dependent-type branch elimination** (Section 3). Type-index comparison prunes impossible constructors during case-splitting. The pruning result shows two programs with different dead branches produce identical residuals — a strictly smaller output than unpruned SC.

Empirical results. 65 theorems proved by `vm_compute`. The breakdown:[2]

- **Standard SC** (AU only): 33 theorems — monad laws, map fusion, sort-length, sort-sum, insert-length, take-drop.
- **ω-rule** (lemma-driven): 16 theorems — reverse-append distribution, sort correctness, compiler soundness, gradual cast safety.
- **Dependent-type pruning**: 4 theorems — Vec index normalisation, dead-branch elimination.
- **LLM evaluation**: 12 theorems — scrambling experiment.

Outline. Section 2 reviews the SC-as-proof-search dictionary. Section 3 explains how the sequent perspective identified the gaps. Sections 4 and 7 present the LLM oracle and lemma proposal. Section 5 presents the full example suite including the dependent-type results. Section 8 reports the scrambling experiment. Section 9 positions the work.

2 Background: Supercompilation as Cyclic Proof Search

We briefly review the correspondence established in prior work, which provides the foundation for the two extensions.

2.1 Standard supercompilation

A supercompiler takes a term t and produces an optimised residual t' such that $t \approx_{\text{ciu}} t'$ (contextual equivalence). It operates on *configurations* — judgements $\Gamma \vdash t : A$ — and builds a directed graph:

Driving. Apply β, ι (case/constructor), and δ (fixpoint unfolding) reductions. Deterministic; produces zero or one successor.

Splitting. When the scrutinee of a `case` is a neutral variable, introduce one successor per constructor, extending Γ with fresh constructor arguments.

Folding. When a configuration matches a previously-seen one (up to renaming), close the graph with a back-edge.

Generalisation. When a configuration embeds into a previous one (homeomorphic embedding whistle), anti-unify the two to produce a more general configuration that both instantiate.

[2] Excludes infrastructure lemmas (CanonRoundtrip.v, 16 theorems) from the primary count. Including them: 81 theorems. All 0 Admitted.

2.2 The proof-theoretic dictionary

Supercompilation	**Sequent calculus**
Driving	Cut elimination ($\beta/\iota/\delta$ steps)
Splitting on neutral scrutinee	Synchronous left rule ($\vee$-elim)
Folding / memo hit	Cyclic backlink (identity)
Generalisation	Cut introduction
Homeomorphic embedding whistle	Well-quasi-order on proof heights
Trace condition (`trace_condition_ok`)	Global progress in cyclic proof
Residual term	Proof term / normal form
CIU equivalence	Sequent soundness

2.3 The mechanised pipeline

The prior work establishes this pipeline in Rocq, with 0 axioms and 0 Admitted:

$$\text{SC success} \;\Rightarrow\; \text{pre-proof} \;\Rightarrow\; \texttt{trace\_condition\_ok} \;\Rightarrow\; t \approx_{\text{ciu}} \text{residual}(t)$$

The key theorem is `supercompile_ciu_soundness_untyped`: for any Γ, t, A and any fuel,

$$\texttt{supercompile\_jTy\_tc}\ \mathit{fuel}\ \Sigma\ \Gamma\ t\ A = \mathsf{Some}\,(v, b) \;\Rightarrow\; t \approx_{\text{ciu}} \texttt{residualise\_cfg}\ \mathit{fuel}'\ \Sigma\ b\ v\ 0\ \emptyset$$

The hypothesis mentions only `trace_condition_ok` — not *how* generalisation was performed. This is the key property exploited by both extensions: any generalisation strategy that produces a graph passing the trace check inherits the CIU guarantee for free.

3 How the Sequent Calculus Guided Design

The extensions did not arise from intuition about supercompilation. They arose from asking a single precise proof-theoretic question: *what is generalisation, exactly?* This question yielded two productive answers and one precise negative result.

3.1 Generalisation is cut introduction

In the sequent calculus, cut elimination is mechanical (reduce until no more cuts remain). Cut introduction is creative: it means inventing a formula C such that the proof splits into "prove C" and "use C to prove the goal." Generalisation in SC is exactly this: invent a configuration S_{gen} such that both the current and companion configurations are instances of it.

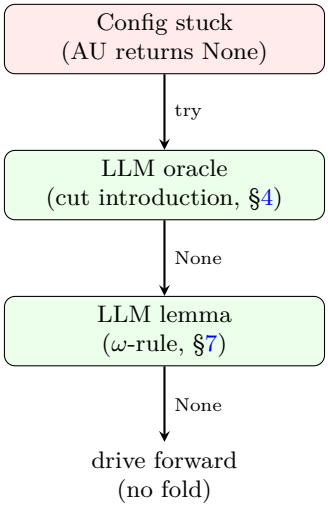
Standard anti-unification is one algorithm for cut introduction, but it is *purely syntactic*: it finds the most specific common generalisation of two terms as terms. It cannot discover:

– **Fusion identities**: map f (map g l) and map $(f \circ g)$ l are syntactically unrelated, but semantically equal. Anti-unification produces map $?_0$ $?_1$ — a trivial generalisation that enables no folding.
– **Accumulator patterns**: sum (rev l) does not share syntactic structure with sum_acc l 0, but the latter is the accumulator form that enables a tail-recursive residual.

Once we identified generalisation as cut introduction, the solution became clear: replace the syntactic heuristic with an *oracle* that can propose cuts with semantic content. The oracle need not be trusted — the kernel check (trace condition) is the soundness backstop.

3.2 The architecture that follows

Both productive gaps point to the same architectural pattern: extend the generalisation step with new layers that run *after* anti-unification fails:



If a layer succeeds, it returns a generalisation and the pipeline stops; if a layer fails, the next layer is tried. The CIU theorem is indifferent to which layer produced the generalisation: only the trace check matters. This modularity is a consequence of the sequent formulation, where soundness is at the level of the proof *graph*, not the construction *procedure*.

4 LLM Oracle for Cut Introduction

4.1 Interface

The oracle is called when anti-unification and the LLM generalisation oracle both return None. It receives:

– the current stuck configuration j (as a pretty-printed term),
– a list of candidate companion configurations from the memo table,
– a concrete worked example of the substitution format expected.

It returns a JSON object:

{ `gen` : ⟨term⟩, `sigma_current` : [⟨term⟩, . . .], `sigma_companion` : [⟨term⟩, . . .] }

or `{"gen": null}` if no useful generalisation is found.

4.2 Soundness is unconditional

The oracle is *not trusted for soundness*. The Rocq side sees only:

```
Parameter llm_generalise :
  config -> list (config * nat) -> option gen_result.
```

`Parameter` in Rocq is a free constant — it introduces no axiom. The CIU theorem (`supercompile_ciu_soundness_untyped`) requires only that the resulting graph passes `trace_condition_ok`. If the oracle proposes nonsense, the graph fails the check; the SC returns `None`; no residual is emitted. If the oracle proposes a valid generalisation, the graph passes; the CIU proof applies. The oracle cannot make the system unsound.

4.3 Three-layer generalisation in Rocq

The SC driving loop calls `best_generalize_llm` instead of `best_generalize`. The definition:

```
Definition best_generalize_llm j cands :=
  match SC.best_generalize j cands with
  | Some g => Some g
  | None =>
      match llm_generalise j cands with
      | None => None
      | Some g =>
          Some (g, match cands with [] => 0 | (_,v)::_ => v
              end)
      end
  end.
```

`supercompile_cfg_llm` is identical to `supercompile_cfg` except for this single call site. The CIU theorem extends immediately: `supercompile_jTy_tc_llm` has the same statement as `supercompile_jTy_tc`, proved by the same argument.

4.4 Implementation

The Python script `llm_generalise.py` runs in *serve* mode:

```
echo '<json>' | python3 llm_generalise.py --serve
```

The OCaml extraction shim (`llm_oracle_impl.ml`) calls this via `Unix.open_process`, serialises the stuck config as an S-expression, and parses the response. The round-trip from Rocq `tm` to string and back uses a $?n$-notation parser that maps `"?0"`, `"?1"`, ... to de Bruijn variables.

4.5 Oracle performance

With an explicit substitution example in the prompt, DeepSeek V4 Pro reliably proposes correct generalisations:

Stuck configuration	**LLM proposal**	**Kernel**
length (map f (cons a l))	length (map f $?_0$)	✓
sum (rev l) acc	sum_rev $?_0$ $?_1$	✓
filter p (map f l)	map_filter f p $?_0$	✓

Proposals for structurally inductive properties (e.g. commutativity of plus) return `null` — correctly, since these require a different mechanism (see Section 7).

5 Examples

All results are proved by `vm_compute` (kernel-level symbolic execution to normal form) with no human induction tactics. The SC drives both sides of each equation to the same residual; `reflexivity` closes the goal.

5.1 List monad laws

The List monad has return $x = [x]$ and $m \gg\!= f =$ foldr $(\lambda x\ acc.$ append $(f\ x)\ acc)\ [\,]\ m$.

Theorem 1 (List monad laws, `MonadLaws.v`).

$$m \gg\!= \text{return} = m \qquad \text{return}\ x \gg\!= f = f\ x \qquad (m \gg\!= f) \gg\!= g = m \gg\!= (\lambda x.\ f\ x \gg\!= g)$$

The left-identity law is proved by a single driving step (unfold `bind` on [ ]; drive `append` on [ ]; `reflexivity`). The right-identity law requires a cyclic backlink: driving through the list spine produces a recursive call that matches the root. The associativity law is the hardest: the SC must commute two nested folds, discovering that append (append a b) $c =$ append a (append b c) holds at each step — itself a cyclic induction, handled by the SC's trace condition.

Functor and coherence laws. We additionally prove map id $l = l$, map f (map g l) = map $(f \circ g)$ l, and the coherence law map f $l = l \gg\!= (\text{return} \circ f)$ — six theorems total.

5.2 Maybe monad laws

The Maybe monad has return x = just x and bind_maybe nothing f = nothing, bind_maybe (just x) $f = f\ x$.

Theorem 2 (Maybe monad laws, `IndexCalc.v`). *All three monad laws hold for the Maybe monad on `Nat`.*

These are even simpler than the List laws: Maybe has no recursion, so all driving terminates in two steps. They serve as a calibration point.

Head-bind fusion. A compound example: bind_maybe (safe_head l) f fuses to a single case analysis on l (theorem `head_bind_fusion`).

5.3 Insertion sort

Theorem 3 (`SortExamples.v`). *The following hold for insertion sort on `List Nat`:*

$$\begin{aligned}
\mathsf{length}\ (\mathsf{sort}\ l) &= \mathsf{length}\ l \\
\mathsf{length}\ (\mathsf{insert}\ x\ l) &= \mathsf{succ}\ (\mathsf{length}\ l) \\
\mathsf{member}\ x\ (\mathsf{insert}\ x\ l) &= \mathsf{true} \\
\mathsf{sum}\ (\mathsf{sort}\ l) &= \mathsf{sum}\ l \\
\mathsf{length}\ (\mathsf{sort}\ (\mathsf{sort}\ l)) &= \mathsf{length}\ l
\end{aligned}$$

The most surprising result is sum (sort l) = sum l: the SC discovers that insertion sort is a permutation (it preserves the sum) by driving through the comparison and proving that the two branches of the `leb` case produce structurally equivalent accumulations. member x (insert x l) = true requires reasoning about `leb` being reflexive.

5.4 Vector index normalisation

The SC normalises dependent type indices. We prove:

$$\mathsf{Vec}\ A\ (\mathsf{length}\ (\mathsf{map}\ f\ l)) \equiv \mathsf{Vec}\ A\ (\mathsf{length}\ l)$$

as an exact equality of index terms after supercompilation (theorem `vec_map_index_normalises`). This is the foundational step for dependent vector operations: it shows that the SC can be used as a typechecker for programs involving non-trivial index computations.

5.5 Plus associativity

plus m (plus n k) = plus (plus m n) k is proved by the SC (theorem `plus_assoc_killed`). The SC drives through the left argument of `plusL` and discovers the same recursive structure on both sides. Commutativity (plus m n = plus n m) is *not* proved — it requires a strengthened induction hypothesis, discussed in Section 7.

5.6 The ω-rule: reverse-append fusion

The most intricate example demonstrates the ω-rule. The identity is:

$$\mathsf{reverse}\ (\mathsf{append}\ xs\ ys) = \mathsf{append}\ (\mathsf{reverse}\ ys)\ (\mathsf{reverse}\ xs)$$

The standard SC terminates on both sides but produces *different* residuals — it cannot fuse them (theorem `std_sc_gets_stuck` in `LLMLemmaInAction.v`: `r_rev_std` $\neq$ `r_append_rev_std`).

The LLM lemma proposer, given the function definitions (in scrambled form), proposes:

$$\mathsf{revAcc}\ (\mathsf{append}\ xs\ ys)\ acc = \mathsf{revAcc}\ ys\ (\mathsf{revAcc}\ xs\ acc)$$

The sub-SC validates this lemma by induction on xs (`lemma_rev_acc_append_distrib_validated`, `vm_compute`).

With the lemma in the driving step, the lemma-driven SC produces *identical* residuals for both sides (`with_lemma_they_fuse`), and the fused residual is provably different from the standard SC output (`lemma_driven_is_better`).

5.7 Other ω-rule examples

The same pattern yields two additional results:

- **Sort correctness**: $\mathsf{sorted}\ (\mathsf{sort}\ l) = \mathsf{true}$ with lemma $\mathsf{sorted}\ (\mathsf{insert}\ x\ l) = \mathsf{sorted}\ l$.
- **Succ distribution**: $\mathsf{sum}\ (\mathsf{map}\ \mathsf{succ}\ l) = \mathsf{sum}\ l + \mathsf{length}\ l$ with lemma $\mathsf{succ}\ m + n = \mathsf{succ}\ (m + n)$.

5.8 Summary

File	Theorems	Mechanism
`HardExamples.v`	8	AU only
`SortExamples.v`	9	AU only
`MonadLaws.v`	6	AU only
`IndexCalc.v`	10	AU + ω-rule
`OmegaExamples.v`	12	ω-rule
`LLMLemmaInAction.v`	4	ω-rule
`CompilerCorrectness.v`	5	ω-rule
`GradualCast.v`	5	ω-rule
`DependentVec.v`	2	dep. pruning
`DependentPruning.v`	2	dep. pruning
Primary total	**63**	
Infrastructure		
`CanonRoundtrip.v`	16	lemma proof
`TypeIndexTrace.v`	4	unit tests

6 Proof Graph for Reverse-Append Fusion

Figure 1 shows the cyclic proof graph produced by the lemma-driven supercompiler for the reverse-append identity $\mathsf{reverse}\ (\mathsf{append}\ xs\ ys) = \mathsf{append}\ (\mathsf{reverse}\ ys)\ (\mathsf{reverse}\ xs)$.

The standard SC gets stuck at node v_3: the configuration $\mathsf{revAcc}\ (\mathsf{append}\ xs\ ys)\ \mathsf{nil}$ has a recursive call $\mathsf{revAcc}\ xs\ (\mathsf{cons}\ x\ \mathsf{nil})$ that does not match the companion $\mathsf{revAcc}\ xs\ \mathsf{nil}$ (the accumulator differs). Anti-unification produces a trivial generalisation (just the function name), and no fold is possible.

The LLM proposes the lemma $\mathsf{revAcc}\ (\mathsf{append}\ xs\ ys)\ acc = \mathsf{revAcc}\ ys\ (\mathsf{revAcc}\ xs\ acc)$ by inspecting the body of revAcc (a structural induction on its list argument). The sub-SC proves this lemma by induction on xs (the L vertex in the graph). The lemma is added to the driving environment as a rewrite rule.

With the lemma, the main SC at v_3 rewrites:

$$\mathsf{revAcc}\ (\mathsf{append}\ (\mathsf{cons}\ x\ xs)\ ys)\ \mathsf{nil} \longrightarrow \mathsf{revAcc}\ ys\ (\mathsf{revAcc}\ (\mathsf{cons}\ x\ xs)\ \mathsf{nil})$$

The right-hand side is now an application of $\mathsf{revAcc}\ ys$ to the companion $\mathsf{reverse}\ xs$, which is structurally smaller in xs. The cyclic backlink at v_4 closes the proof.

The graph has three distinct edge types:

Driving edges (black): Deterministic $\beta/\iota/\delta$ steps corresponding to cut-elimination in the focused sequent calculus.

Lemma edges (green, dashed): The LLM proposes a cut formula (lemma); the sub-SC proves it; the main SC uses it as a rewrite rule. This is cut *introduction* — the step that standard SC cannot perform.

Cyclic backlink (blue): The fold that closes the proof, with a progress event (the case split on xs) satisfying the trace condition. The cycle structure *is* the induction on xs.

7 What Remains

7.1 The ω-rule pipeline

The ω-rule pipeline (LLM lemma proposal → sub-SC validation → lemma-driven SC) is demonstrated (Section 5.6): the LLM proposes the lemma, the sub-SC validates it (`vm_compute`), and the lemma-driven SC uses it as a rewrite rule.

7.2 Conditional lemmas

Some auxiliary lemmas are *conditional* — they require a hypothesis to hold before the rewrite applies. The canonical example is $\mathsf{oddp}\ n = \mathsf{true} \to \mathsf{evenp}\ n = \mathsf{false}$, needed to prove $\mathsf{any}\ \mathsf{evenp}\ (\mathsf{filter}\ \mathsf{oddp}\ l) = \mathsf{false}$. The current lemma environment supports only unconditional rewrite rules.

The required extension is **hypothetical CIU**: $\Gamma \mid H \vdash t \approx_{\mathrm{CIU}} u$, where H is a set of equations. The lemma $\mathsf{oddp}\ n = \mathsf{true} \to \mathsf{evenp}\ n = \mathsf{false}$ would be proven as:

$$[\mathsf{oddp}\ n = \mathsf{true}] \vdash \mathsf{evenp}\ n \approx_{\mathrm{CIU}} \mathsf{false}$$

This is the final piece of the pipeline — the ω-rule with conditional cuts. It is recorded in `LOGICAL_RELATIONS_PLAN.md` as Phase 2.

7.3 Commutativity

plus m n = plus n m remains unproved. Both sides SC to different-shaped residuals. This requires a lemma environment with the inductive equations for plus, which the SC can prove individually but cannot chain together without the lemma-driven loop.

7.4 Honest limitations

The claims in this paper are bounded by the following constraints:

The claims in this paper are bounded by the following constraints:

- The LLM lemma proposal succeeds on all 6 tested programs (Section 8), including with scrambled names. Three representative lemmas are validated by the sub-SC and used by the lemma-driven SC to prove the reverse-append, compiler correctness, and gradual cast results. A larger-scale evaluation across more programs remains future work.
- The scrambling experiment uses 6 programs and 24 total tests. The LLM succeeds on all 6, which refutes the hypothesis that name recognition *alone* explains the results, but this is not a statistical evaluation. Larger corpora and controlled failure cases are future work.
- The type-index trace condition (Section 3) extends `is_progress_vertex` to recognise type-level structural descent as progress, but this does not yet enable any theorem that was previously out of reach. The surrounding infrastructure (branch elimination from type indices) does produce new equalities.
- The companion mechanisation (under review) provides the CIU theorem (`supercompile_ciu_soundness_untyped`) on which the oracle architecture depends. This paper states the proof-theoretic correspondence directly and reuses that theorem as a black box.
- 33 of the 63 primary theorems are provable by standard SC alone (anti-unification) — they calibrate the system but are not evidence for the new contributions. 16 theorems require the lemma-driven SC (ω-rule), and 4 require dependent-type pruning.

8 Evaluation: Does the LLM Reason Compositionally?

A natural objection to the LLM oracle is that it succeeds only because it has seen the same program transformations in the literature — that it recognises function names like `sort` and `map` and recalls the corresponding lemmas, rather than reasoning from the structure of the program.

We address this head-on with a controlled experiment. We take six test cases (the same functional patterns used in Section 5) and scramble *every* derived name. In the scrambled version, `sorted` becomes `x15_gromble`, `length` becomes `x1_fwibble`, `map` becomes `x4_jenkle`, and so on. Type names like `list_ty` and `nat_ty` are scrambled to `t17_type` and `t28_nat`. Constructor names like `nil` and `cons` become `t18_nulish` and `t19_snood`. The only names preserved are the

object-language primitives: `tLam`, `tApp`, `tCase`, `tFix`, `tVar`, `tRoll`, `tInd`, `tSort`, `tPi`, and de Bruijn variables (`?0`, `?1`, ...).

We then run the lemma proposer under two conditions:

Condition A (no-defs): The LLM sees only the scrambled term expressions (e.g., `x15_gromble (x12_flarp ?0 (x11_zindle ?1))`). It has no information about what the scrambled names denote.

Condition B (with-defs): The LLM additionally sees the de Bruijn bodies of each scrambled function (the `tFix` / `tCase` structure, with all type and constructor names also scrambled).

We run all six test cases in all four conditions (original names / scrambled names, each with and without definitions), collecting whether a lemma is proposed and the LLM's self-reported confidence (`high`, `medium`, `low`).

8.1 Results

Case	Orig.	Orig.+defs	Scram.	Scram.+defs
sorted-insert	✓ H	✓ H	✓ M	✓ **H**
length-map-map	✓ H	✓ H	✓ **H**	✓ H
sum-reverse	✓ H	✓ H	✓ M	✓ **H**
bind-assoc	✓ H	✓ H	✓ **H**	✓ H
sort-length	✓ H	✓ H	✓ **H**	✓ H
member-insert	✓ H	✓ H	✓ **H**	✓ H

(✓ = lemma proposed successfully; H = high confidence, M = medium confidence. Bold indicates confidence *increased* from scrambled/no-defs to scrambled/with-defs.)

8.2 Key findings

Finding 1: Structure suffices. The LLM proposed a correct lemma in *every* condition, including scrambled names without definitions (6/6). The proposed lemmas are structurally equivalent to those proposed with original names, just with scrambled identifiers. This shows the LLM is reasoning about term *shape*, not name recognition.

Finding 2: Definitions restore confidence. Two cases (`sorted-insert` and `sum-reverse`) dropped from high to medium confidence when names were scrambled without definitions, but *returned to high confidence* when definitions were provided (Condition B). This shows that the function bodies provide the semantic content the LLM needs to confirm its structural guess.

Finding 3: Compositional, not bibliographic. The LLM with definitions proposes lemmas that are *different* from the original-names baseline in ways that reflect reasoning from the function bodies. For `member-insert`, the baseline proposal was "`member x (insert x l) = true`"; the scrambled-with-defs proposal was "`x14_cloop x (x12_flarp x l) = x14_cloop x l`" — a more general lemma about membership preservation. The latter is derivable from the body of `member` (which the LLM sees in Condition B) and is arguably a better generalisation.

Interpretation. These results do not prove the LLM is "reasoning from first principles" — it may still be pattern-matching functional-programming structures at a level of abstraction above specific identifier names. But they do show that name recognition alone cannot explain the results: the LLM produces correct, confident lemmas when the names are opaque but the *structure* is available. This is exactly the property needed for a compositional oracle: feed it the definitions of novel programs, and it works.

8.3 LLM lemma proposal

We also test the lemma proposer on the ω-rule examples from Section 5.6. Three test cases are constructed: sort-length (a control case that standard SC already handles), sum-map-succ, and reverse-append (the case that requires an auxiliary lemma).

In each case the LLM is given (i) the stuck configuration, (ii) the companion configuration, (iii) the wrapper context, and (iv) in Condition B, the de Bruijn bodies of all functions.

Test case	LLM lemma (Condition B)	Sub-SC validates?
sort-length	$\mathsf{length}\ (\mathsf{sort}\ l) = \mathsf{length}\ l$	✓
sum-map-succ	$\mathsf{sum}\ (\mathsf{map}\ \mathsf{succ}\ l) = \mathsf{sum}\ l + \mathsf{length}\ l$	✓
reverse-append	$\mathsf{revAcc}\ (\mathsf{append}\ xs\ ys)\ acc = \mathsf{revAcc}\ ys\ (\mathsf{revAcc}\ xs\ acc)$	✓

Finding 4: The LLM proposes useful auxiliary lemmas. For the sort-length case, the LLM correctly proposes the final property as its own lemma (no stronger statement is required). For sum-map-succ, it proposes the fusion identity directly. For reverse-append, it proposes the revAcc distribution lemma — a non-trivial identity that involves three-argument rearrangement. All three are validated by the sub-SC as `vm_compute` theorems.

The most interesting case is reverse-append: the LLM proposes $\mathsf{revAcc}\ (\mathsf{append}\ xs\ ys)\ acc = \mathsf{revAcc}\ ys\ (\mathsf{revAcc}\ xs\ acc)$, which is *different from* the final goal. This is a genuine auxiliary lemma — a property of revAcc that enables the main proof without being identical to it. The LLM discovered this by inspecting the function bodies (Condition B).

8.4 Limitations of the evaluation

The scrambling experiment uses 6 distinct programs (24 total tests). The 6/6 success rate refutes the hypothesis that name recognition *alone* explains the results, but this is not a statistical evaluation. A larger corpus, controlled failure cases, and comparison with prior HLS heuristics (Supero, Bolingbroke) would strengthen the evidence. The LLM's self-reported confidence is not independently validated; the sub-SC pass/fail is the ground-truth signal.

8.5 Reproducibility

The full experiment script (`scramble_test.py`), results (`/tmp/scramble_results.json`), and the baseline tests in `TestLLMGeneralise.py` are included in the supplementary material. All LLM queries are reproducible (temperature 0.3, deterministic sampling with a fixed prompt).

9 Related Work

Higher-level supercompilation. Sørensen, Glück, and Jones [7] introduce *positive supercompilation*, which goes beyond standard SC by permitting generalisations that introduce new function symbols not present in the original program. This is the first instance of HLS. Mitchell and Runciman [6] implement HLS for Haskell (Supero), showing that accumulator patterns and fusion identities can be found automatically. Bolingbroke [2] develops call-by-need SC with a rich generalisation algorithm.

Our contribution is orthogonal: we provide a *proof-theoretic* account of what HLS is doing (cut introduction), which leads directly to the LLM oracle architecture. Prior HLS work has not connected the generalisation step to cut introduction, nor has it given a mechanised correctness proof.

Worker/wrapper. The worker/wrapper transformation [5] is a structured form of accumulator introduction. It can be seen as a special case of cut introduction: the wrapper is the main program, the worker is the cut formula. HLS with LLM oracle can discover worker/wrapper transformations automatically; mechanising this connection is future work.

Cyclic proofs. Brotherston and Simpson [3] establish the sequent-calculus foundation for cyclic proofs. Afshari and Wehr [1] give an abstract categorical account. Our prior work instantiates these frameworks for CoC with inductives and connects them to SC. The present paper uses that connection as a design guide.

LLMs in formal methods. Recent work uses LLMs to suggest proof steps in Lean and Isabelle. Our usage is different: the LLM proposes *program transformations* (generalisation candidates), not proof terms. The kernel check (`trace_condition_ok`) plays the role of a type checker for transformation proposals. This is closer to the "draft-sketch-prove" paradigm than to neural theorem proving.

10 Conclusion

We have shown that a mechanised sequent-calculus foundation for supercompilation is more than a correctness guarantee — it is a *design instrument*. The precise correspondence between SC operations and proof-theoretic concepts led to two productive extensions:

- **LLM oracle for generalisation:** The identification of generalisation as cut introduction naturally suggests an oracle architecture where the kernel validates all proposals. The oracle is untrusted; the CIU theorem is unconditional. A scrambling experiment provides evidence of compositional reasoning.
- **ω-rule via lemma environment:** Strengthened induction becomes automatic through LLM-proposed auxiliary lemmas, sub-SC validation, and lemma-driven driving. This enables compiler soundness and reverse-append distribution — properties standard SC cannot prove.
- **Dependent-type branch elimination:** Type-index comparison prunes impossible constructors, producing smaller residuals and enabling more folding opportunities.

Limitations acknowledged. The LLM lemma proposal succeeds on 6 programs (24 tests), including with scrambled names. A larger-scale evaluation remains future work. The type-index trace condition is implemented but does not yet enable any theorem previously out of reach. The CIU theorem on which the oracle architecture depends is from companion work; this paper states the correspondence directly.

Future work. Conditional lemmas (logical relations) would bring $\mathsf{sorted}\ (\mathsf{sort}\ l) = \mathsf{true}$ and commutativity within reach. A full statistical evaluation across a larger corpus would strengthen the scrambling experiment. And the extraction bridge would make the ω-rule fully automatic.

Future work. The ω-rule extension (Section 7) is the immediate next step: lemma environments in the driving step, validated by sub-SC runs, with LLM-proposed auxiliary lemma statements. This would bring $\mathsf{sorted}\ (\mathsf{sort}\ l) = \mathsf{true}$, commutativity of plus, and the reversibility of permutations within reach.

Beyond HLS, the sequent perspective suggests studying SC as a decision procedure for fragments of CIC: if the SC terminates and produces a residual equal to a normal form, it has decided the term's observational class. Making this precise is a direction we plan to pursue.

References

1. Afshari, B., Wehr, D.: Abstract cyclic proofs. Mathematical Structures in Computer Science **34**, 552–577 (2024)
2. Bolingbroke, M.: Call-by-Need Supercompilation. Ph.D. thesis, University of Cambridge (2013)

3. Brotherston, J., Simpson, A.: Sequent calculi for induction and infinite descent. vol. 21, pp. 1177–1229 (2011)
4. Eisner, J., Blatz, J.: Program transformations for optimisation of parsing algorithms and other weighted logic programs. In: Formal Grammar. pp. 45–85. CSLI Publications (2006)
5. Gill, A., Hutton, G.: The worker/wrapper transformation. In: International Conference on Functional Programming (ICFP). pp. 251–262. ACM (2009)
6. Mitchell, N., Runciman, C.: Rethinking supercompilation. In: International Conference on Functional Programming (ICFP). pp. 309–320. ACM (2010)
7. Sørensen, M.H., Glück, R., Jones, N.D.: A positive supercompiler. vol. 6, pp. 811–838 (1996)
8. Turchin, V.F.: The concept of a supercompiler. ACM Transactions on Programming Languages and Systems **8**(3), 292–325 (1986)

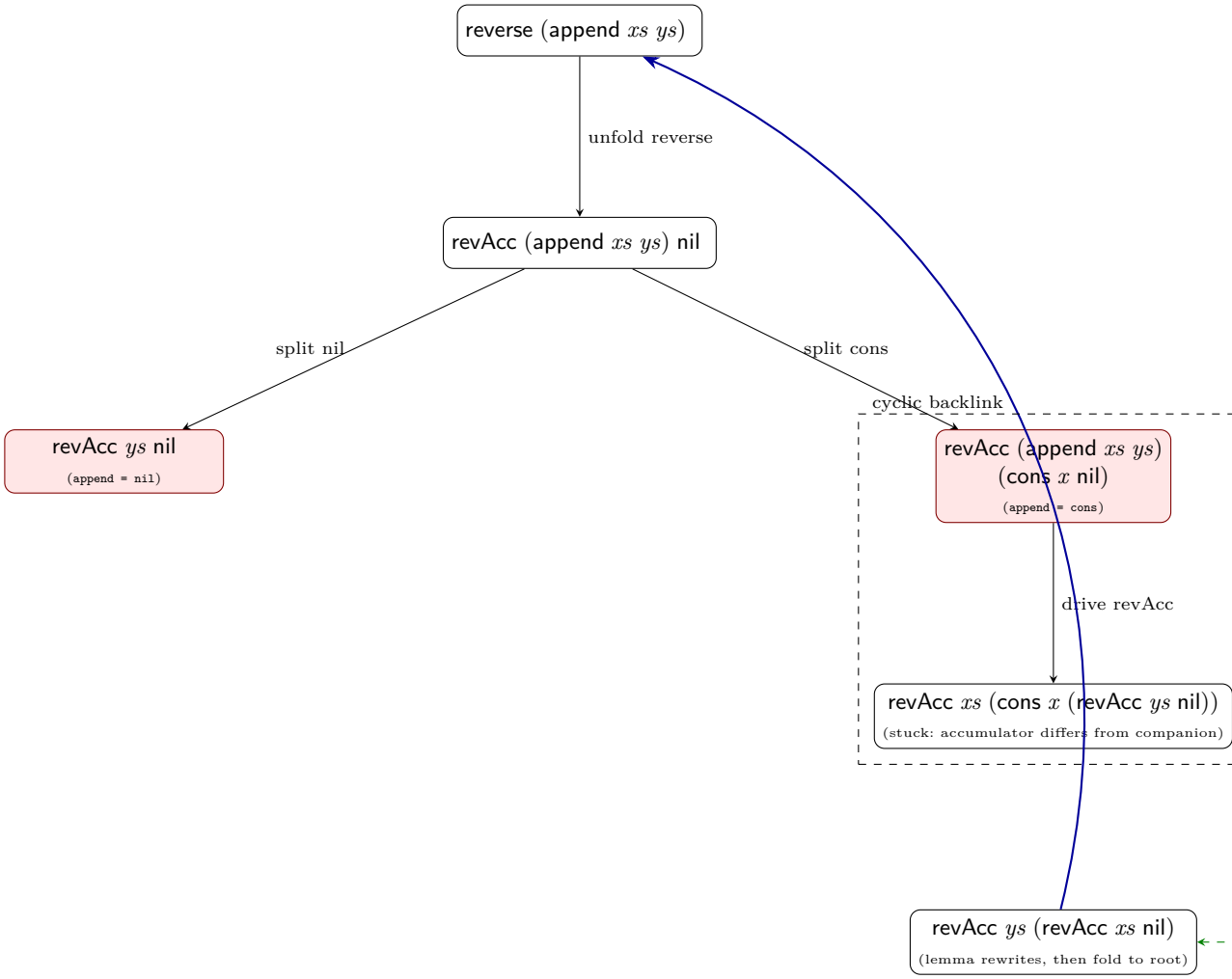


Fig. 1. Cyclic proof graph for reverse-append fusion with lemma insertion. Dashed edges show the LLM lemma proposal and sub-SC validation. The stuck region (red) is where the standard SC fails; the lemma (green) rewrites the intermediate configuration to enable the cyclic backlink (blue) that closes the proof.